Bilan des TP Web n°1 à 5bis

HACHE – Polytech Lyon 3A – 2026-2027

TP 1 – HTML : construire le site

On a fait quoi ?

On a créé la base du site « Initiation au développement Web » : un en-tête, un menu à gauche, un contenu à droite et un pied de page. Trois pages : `index.html` (accueil), `bases.html` (les bases du HTML) et `tableau.html` (les tableaux). Chaque page doit passer le validateur W3C sans erreur.

HTML

- `<!DOCTYPE html>` + `<html lang="fr">` : document HTML5 en français.
- `<meta charset="utf-8">` : les accents s'affichent correctement.
- `<link rel="stylesheet" href="tphtml.css">` : relie la feuille de style.
- `<header>`, `<nav>`, `<article>`, `<footer>` : balises **sémantiques**, elles disent ce que contient chaque zone.
- `<h1>` … `<h6>` : titres. **Un seul** `<h1>` par page, puis `<h2>` pour les parties.
- `<p>`, `<br>`, `<hr>` : paragraphe, saut de ligne, ligne horizontale.
- `<ul>` / `<ol>` + `<li>` : liste à puces / liste numérotée.
- `<b>`, `<i>`, `<u>`, `<del>`, `<small>` : gras, italique, souligné, barré, petit.
- `<pre><code>` : code affiché tel quel (espaces et retours à la ligne gardés).
- `<abbr title="...">` : abréviation avec info-bulle.
- `<` et `>` : afficher les caractères < et >.
- `<a href="#listes">` + `id="listes"` : lien vers une partie de la page (le sommaire de `bases.html`).
- `<table>`, `<tr>`, `<th>`, `<td>` : tableau, ligne, cellule d'en-tête, cellule.
- `colspan="2"` / `rowspan="2"` : une cellule s'étend sur 2 colonnes / 2 lignes.
- `<img src="..." alt="...">` : image avec un texte alternatif **obligatoire**.

```
<body>
    <header>Initiation au développement Web</header>
    <div class="contenu">
        <nav>
            <a href="index.html" class="clic">Accueil</a>
            <a href="bases.html">Les bases du langage HTML</a>
        </nav>
        <article>
            <h1>Les tableaux</h1>
        </article>
    </div>
    <footer>Copyright 2024 - 2025 Polytech Lyon 3A Info</footer>
</body>
```

CSS (`tphtml.css`)

```
* { box-sizing: border-box; } /* la largeur inclut padding + bordure */
```

```
div.contenu { display: flex; } /* menu et contenu côte àcôte */
nav { flex: 25%; display: flex; flex-direction: column; }
article { flex: 75%; }

nav a.clic { background-color: #175; color: white; } /* page active */

@media (max-width: 600px) { /* petit écran : on empile */
    div.contenu { flex-direction: column; }
}
```

Le point important

La validité W3C. Le navigateur affiche même du HTML faux, donc les erreurs ne se voient pas : il faut passer chaque page sur `validator.w3.org`. Les erreurs que j'ai dû corriger :

- un seul `<body>` par page (j'en avais deux, refermés trop tôt) ;
- un `id` ne contient **pas d'espace** (`id="formatage"`, pas `id="Formatage du texte"`) ;
- on ne met pas `<ul>` ou `<pre>` dans un `<p>` ;
- toute `<img>` a un `alt` ;
- pas d'attribut `border` sur `<table>` (obsolète) : c'est le CSS qui fait les bordures.

TP 2 – CSS : mettre en forme une page

On a fait quoi ?

On reçoit `basesCSS.html` **sans aucun style**. On écrit `style.css` pour obtenir le rendu de la maquette, sans toucher au HTML (sauf la ligne `<link>`). Ensuite on affiche cette page dans le site grâce à une `<iframe>`. Dans une 2e page, une iframe montre la même page HTML avec 4 feuilles de style différentes.

HTML

- `<link rel="stylesheet" href="style.css">` : seule ligne ajoutée à `basesCSS.html`.
- `<iframe src="..." title="...">` : affiche une page HTML dans une autre. La taille se règle en CSS (`width="100%"` est invalide en HTML5).

CSS

```
h1 { color: red; text-align: center; font-variant: small-caps; } /* petites majuscules */
h2 { background-color: red; border-left: 50px solid black;
     text-transform: capitalize; } /* Majuscule ÀChaque Mot */
article { border: 2px dotted gray; } /* pointillés */

#info { width: 600px; margin: auto; border-radius: 10px; } /* id = # */
.documentation { background-color: green; color: white; } /* classe = . */
.documentation a { text-decoration: none; color: greenyellow; }
.documentation a:hover { background-color: lightgreen; } /* survol */

table { width: 60%; margin: auto; }
tr { height: 2em; }
th { background-color: darkblue; color: white; height: 3em; }
tr:nth-child(even) { background-color: lightblue; } /* lignes paires */
tr:last-child { color: blue; } /* dernière ligne */
```

Les sélecteurs à connaître : `p` (balise), `.classe`, `#id`, `.documentation a` (un `a` *dans* `.documentation`), `a:hover`, `tr:nth-child(even)`, `tr:last-child`.

Le point important : `margin: auto`

`margin: auto` **centre** un bloc : le navigateur partage l'espace restant à parts égales à gauche et à droite. **Ça ne marche que si le bloc a une largeur** (`width: 600px` pour `#info`, `width: 60%` pour le tableau). Sans largeur, le bloc prend toute la place et il n'y a rien à partager.

Piège

`dotted` = pointillés, `dashed` = tirets. `greenyellow` (vert-jaune) ≠ `yellowgreen`. Et `box-sizing: border-box` au début du fichier, sinon `width: 600px` + `padding: 10px` donne une boîte de 620px.

TP 3 – JavaScript : rendre les pages interactives

On a fait quoi ?

Quatre exercices, chacun avec sa page HTML et son fichier dans `javascript/` :

— **Fibonacci** : calculer le terme de rang n saisi par l'utilisateur ;

— **Range(stop, [start], [step])** : renvoyer le tableau des entiers de `start` (inclus) à `stop` (exclu) avec un pas `step` ;

— **L'ampoule** : un clic l'allume ou l'éteint ;

— **Les planètes** : au survol, afficher une description et l'image de la planète.

HTML

— `<input type="text" id="stop">` + `<label for="stop">` : champ de saisie et son étiquette.

— `onclick="Appeler_Range()"` : appelle une fonction JS au clic.

— `<map>` + `<area shape="circle" coords="...">` : zones cliquables dans une image.

— `onmouseover` / `onmouseout` : la souris entre / sort de la zone.

— `<script src="javascript/exercices.js">` **à la fin du** `<body>` : les éléments existent déjà quand le script s'exécute.

CSS

```
#gifPlanete { display: none; } /* image cachée tant qu'on ne survole rien */
img.ampoule { cursor: pointer; } /* curseur "main" : on voit que c'est cliquable */
```

JavaScript

```
// Lire une saisie et écrire un résultat
const texte = document.getElementById("stop").value.trim();
document.getElementById("affichage_range").textContent = "Range = [" + tab + "]";

// Range : paramètres par défaut + garde
function Range(stop, start = 0, step = 1) {
    if (!Number.isFinite(stop) || !Number.isFinite(start)
        || !Number.isFinite(step) || step <= 0) {
        return [];
```

```
    }
    const resultat = [];
    for (let i = start; i < stop; i = i + step) {
        resultat.push(i);
    }
    return resultat;
}

// Ampoule : search() renvoie -1 si le mot n'est pas trouvé
if (image.src.search("bulboff") !== -1) {
    image.src = "images/pic_bulbon.gif";
    image.alt = "Ampoule allumée";
}

// Planètes : on mémorise le texte de départ UNE fois
const texteParDefaut = document.getElementById("desc").textContent;
function writeDefault() {
    document.getElementById("desc").textContent = texteParDefaut;
}
```

Types JavaScript (exercice console) : `typeof NaN` → `"number"`, `typeof [1,2]` → `"object"`, `typeof null` → `"object"`, `typeof un_truc` → `"undefined"`, `typeof function(){}` → `"function"`.

Le point important : vérifier les saisies

L'utilisateur peut taper n'importe quoi. Sans vérification :

- `Range` avec `step = 0` **bloque la page** : `i` ne change jamais, `i < stop` reste vrai, la boucle ne s'arrête pas ;
- un champ vide donne `Number("") = 0`, donc « Fibonacci(0) = 0 » ;
- `Number("abc")` donne `NaN`.

On teste donc la saisie **avant** de calculer, et on affiche un message clair.

Piège

`==` convertit les types (`"5" == 5` est vrai) : utiliser `===`. Préférer `textContent` à `innerHTML` pour écrire du texte. Une image « vide » ne s'écrit pas `src=""` (invalide) : on la cache en CSS.

TP 4 et TP 5 – Lecture : stockage navigateur et Alpine.js

À retenir

- **Cookie** : petite donnée **envoyée au serveur à chaque requête**, avec une date d'expiration (`expires` ou `max-age`). Options : `path`, `secure` (HTTPS seulement), `samesite` (anti-CSRF), `HttpOnly` (invisible pour JS, anti-XSS).
- **localStorage** : reste dans le navigateur, **pas envoyé au serveur**, permanent. `sessionStorage` : effacé à la fermeture de l'onglet. Les deux ne stockent que des **chaînes**.
- **IndexedDB** : vraie base de données objet, asynchrone, pour beaucoup de données.
- **Alpine.js** : petit framework sans build. On écrit le comportement directement dans le HTML avec des directives `x-...`.

TP 5bis – Alpine.js : la Todo List

On a fait quoi ?

Une Todo List qui permet d'ajouter, cocher et supprimer des tâches, avec des statistiques, un **thème sombre** et un mode **lecture facilitée**. Tout est **conservé après actualisation** grâce à `localStorage`. On a aussi écrit les fonctions `setCookie`, `getCookie` et `deleteCookie`.

HTML (directives Alpine)

- `x-data="monApplication()"` : déclare les données de l'application.
- `:class="{ dark: modeSombre, dyslexie: modeDyslexie }"` : ajoute les classes si la valeur est vraie (`:` = `x-bind`).
- `@click="changerSombre()"` : appelle une fonction au clic (`@` = `x-on`).
- `@submit.prevent="ajouterTache()"` : ajoute la tâche avec le bouton **ou** Entrée, sans recharger la page.
- `x-model="texteSaisi"` : le champ et la variable restent synchronisés.
- `x-for="t in listeTaches" :key="t.id"` : une ligne par tâche (sur un `<template>`).
- `x-show="listeTaches.length === 0"` : affiche « liste vide » si besoin.
- `x-text="compterFaites()"` : affiche une valeur calculée.
- `:aria-pressed` : un lecteur d'écran sait si le mode est activé.

```
<body x-data="monApplication()" :class="{ dark: modeSombre, dyslexie: modeDyslexie }">
    <form @submit.prevent="ajouterTache()">
        <label for="nouvelleTache">Nouvelle tâche :</label>
        <input type="text" id="nouvelleTache" x-model="texteSaisi">
        <button type="submit">Ajouter</button>
    </form>
    <template x-for="t in listeTaches" :key="t.id">
        <li>
            <input type="checkbox" x-model="t.fini" @change="sauvegarderLocal()">
            <span :class="{ barre: t.fini }" x-text="t.nom"></span>
        </li>
    </template>
</body>
```

CSS

```
.dark { background: #1e1e1e; color: #f5f5f5; }
.dyslexie { line-height: 1.8; letter-spacing: 0.05em; word-spacing: 0.12em; }
.dyslexie p, .dyslexie li { max-width: 70ch; font-size: 1.08rem; }
.barre { text-decoration: line-through; color: grey; }
```

JavaScript

```
// Préférence : localStorage ne garde que du texte -> 'oui' / 'non'
modeSombre: localStorage.getItem('themeSombre') === 'oui',

changerSombre() {
    this.modeSombre = !this.modeSombre;
    localStorage.setItem('themeSombre', this.modeSombre ? 'oui' : 'non');
},
```

```
// Tâches : tableau d'objets -> texte JSON, et inversement
sauvegarderLocal() {
    localStorage.setItem('mesTaches', JSON.stringify(this.listeTaches));
},
function chargerTaches() {
    try {
        const taches = JSON.parse(localStorage.getItem('mesTaches'));
        return Array.isArray(taches) ? taches : [];
    } catch (erreur) {
        return []; // données abîmées : on repart d'une liste vide
    }
}
```

Le point important : la persistance avec `localStorage`

- `localStorage` ne stocke **que du texte**. Pour un booléen, on enregistre `'oui'`/`'non'` et on compare à la lecture. Pour un tableau de tâches, on utilise `JSON.stringify` pour écrire et `JSON.parse` pour relire (sinon on obtient `"[object Object]"`).
- **Pourquoi pas un cookie ?** Un cookie est envoyé au serveur à chaque requête : inutile pour une préférence d'affichage. `localStorage` reste dans le navigateur.
- On sépare les **préférences** (thème, lecture) des **données métier** (tâches).

Piège

- `:key` dans `x-for` : sans lui, après une suppression, une case cochée peut passer sur une autre tâche.
- `trim()` avant d'ajouter : sinon une tâche faite d'espaces est acceptée.
- `JSON.parse` plante si les données sont invalides : toujours un `try/catch`.
- `x-show` cache l'élément ; `x-if` le retire du DOM.